



Faculdade Anhanguera superior Bacharelado em

Engenharia de Software

3º semestre

William Carlos Piccirilo Lopes

Portifólio : Relatório de aula prática

Disciplina : Análise Orientado a Objetos

Campinas

2025

William Carlos Piccirilo Lopes

Portifólio : Relatório de aula prática

Disciplina : Análise Orientado a Objetos

Aula prática de Análise Orientado a Objetos apresentado para obtenção de média semestral no curso de Engenharia de Software.

Tutor : Vinicius Camargo Prattes

Professora : Vanessa Matias Leite

Campinas

2025

SUMÁRIO

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	O que é Visual Paradigm?.....	4
2.2	Descrição detalhada do produto.....	4
2.3	Algumas funcionalidades do produto.....	5
2.4	Requisitos do sistema.....	6
2.5	O que é diagrama de classes?.....	7
2.6	Os benefícios de um diagramas de classes.....	7
2.7	Componentes básicos de um diagrama de classes.....	8
2.8	Modificadores de acesso de membro.....	9
2.9	Tipos de diagrama de classes.....	10
3	MÉTODOS	13
3.1	Diagrama de classes para um sistema de locação de veículos.....	13
3.2	Sistema de locação de veículos no Visual Paradigm.....	14
4	RESULTADOS	15
5	CONCLUSÃO	16
6	REFERÊNCIAS	17

1. INTRODUÇÃO

Este relatório apresenta a atividade solicitada na disciplina de Análise Orientada a Objetos, realizada no 3º semestre do curso de Engenharia de Software.

O objetivo desta atividade foi desenvolver um diagrama de classes para um sistema de locação de veículos. Utilizando a ferramenta online do software Visual Paradigm.

Neste relatório, será descrito o passo a passo para o desenvolvimento deste diagrama.

2. DESENVOLVIMENTO

2.1 O que é Visual Paradigm ?

O Visual Paradigm é um pacote de desenvolvimento de software e gerenciamento corporativo, que fornece todos os recursos que você precisa para arquitetura corporativa, gerenciamento de projetos, desenvolvimento de software e colaboração em equipe, tudo em uma única solução!

2.2 Descrição detalhada do produto

O procedimento de guia do **Visual Paradigm** incorpora instruções, referências de entrada e amostras para executar todos os passos com as ferramentas necessárias. Sua equipe pode dar início à EA e ao projeto de gerenciamento com facilidade, independentemente do tamanho dela.

O Visual Paradigm fornece um ambiente colaborativo e automatizado para você gerenciar as partes interessadas, tarefas e agendamento de tarefas com o gráfico PERT, gerando relatórios.

Integre diferentes padrões, frameworks e processos de forma transparente. Sua equipe pode selecionar qualquer combinação para se adequar ao seu caso.

2.3 Algumas funcionalidades do produto

- **Ferramentas UML & SysML**
Capture os requisitos funcionais com o diagrama de caso de uso UML. Cada caso em um diagrama de caso de uso representa um objetivo de negócios de alto nível que produz um resultado mensurável de valores empresariais. Os atores (UML) estão conectados com casos de uso para representar os papéis que interagem com as funções.
- **Ferramentas e diagrama BPMN**
Visualize os fluxos de trabalho empresariais. Comunique ideias de processo de negócios com os diagramas BPMN.
- **UML para código, código para UML**
Gere código fonte do modelo de classe UML ou vice-versa.
- **Ferramentas de gerenciamento de projetos**
Compreenda as suas necessidades de gerenciamento de projetos com PERT, plano de implementação e análise de maturidade.
- **Formatos de saída versáteis**
Exporte seu design em uma gama de formatos de arquivos.
- **Ferramenta de geração de documentos**
Gere documentos completos e profissionais com apenas alguns clicks.
- **Guia de ciclo de vida do gerenciamento de projetos**
Processo passo-a-passo para o gerenciamento de projetos, com instruções, exemplos e ferramentas para o desenvolvimento incremental de entregas.
- **Diversidades**
Outras grandes ferramentas que você poderá encontrar no Visual Paradigm são o suporte a várias línguas, desenvolvimento de plug-ins e mais.

2.4 Requisitos do sistema

Hardware:

- Intel Pentium 4 a 2,0 GHz ou superior
- Mínimo de 2.0 GB de RAM, mas é recomendado 4.0 GB.
- Espaço mínimo de 4 GB de disco (NÃO inclui espaço do projeto).
- Microsoft Windows (XP / Vista / 7 / 8,10), Microsoft Windows Server (2000/2003/2008/2012), Linux, Mac OS X 10.7.3 ou superior

Requisitos IDE (para integração IDE):

- Eclipse 3.5 ou superior
- IntelliJ IDEA 11.0 ou superior
- Android Studio 1.3 ou superior
- NetBeans 6.7 ou superior
- Microsoft Visual Studio 2012, 2013, 2015, 2017

2.5 O que é diagrama de classes ?

Os diagramas de classes são um tipo de diagrama da estrutura porque descrevem o que deve estar presente no sistema a ser modelado. Não importa seu nível de familiaridade com diagramas UML ou de classe, nosso software de UML online foi concebido para ser simples e fácil de usar.

A UML foi criada como um modelo padronizado para descrever uma abordagem de programação orientada ao objeto. Como as classes são os componentes básicos dos objetos, diagramas de classes são os componentes básicos da UML. Os diversos componentes em um diagrama de classes podem representar as classes que serão realmente programadas, os principais objetos ou as interações entre classes e objetos.

2.6 Os benefícios de um diagramas de classes

Diagramas de classes oferecem uma série de benefícios para qualquer organização. Use diagramas de classes UML para:

- Ilustrar modelos de dados para sistemas de informação, não importa quão simples ou complexo.
- Entender melhor a visão geral dos esquemas de uma aplicação.
- Expressar visualmente as necessidades específicas de um sistema e divulgar essas informações por toda a empresa.
- Criar gráficos detalhados que destacam qualquer código específico necessário para ser programado e implementado na estrutura descrita.
- Fornecer uma descrição independente de implementação de tipos utilizados em um sistema e passados posteriormente entre seus componentes.

2.7 Componentes básicos de um diagrama de classes

Os itens da diagramação que compõem um diagrama de classes são:

- **Classe:**

É um elemento abstrato que representa um conjunto de objetos. Nela contém a especificação do objeto, suas características, atributos e métodos.

- **Atributo:**

Basicamente define as características da classe, visibilidade, nome, tipo de dados, multiplicidade, valor inicial e propriedade. A visibilidade pode ser tanto pública(+) ou privada (-). Quando ela é definida como pública significa que outras classes podem ter acesso ao atributo, já quando ela é definida como privada, apenas a própria classe tem acesso. Ela ainda pode ser protegida(#) ou atribuída em pacote(~), onde o atributo é acessado pelo relacionamento da classe com a classe externa.

- **Operação:**

Ela trata da função solicitada a um objeto, contém características como nome, visibilidade e parâmetro.

- **Associação:**

Ela pode conter nome, a multiplicidade e o tipo de navegação, indica de onde partem as informações da classe e para onde elas irão. O diagrama de classes seria um retângulo com três linhas, onde a linha superior contém o nome da classe, na linha do meio os atributos, e na linha inferior os métodos e suas funções.

Notações possíveis:

Tipos	Significado
0..1	Zero ou uma instância. A notação n..m indica n para m instâncias.
0..* ou *	Não existe limite para o número de instâncias.
1	Exatamente uma instância.
1..*	Ao menos uma instância.

Figura.1

2.8 Modificadores de acesso de membro

Todas as classes têm diferentes níveis de acesso, dependendo do modificador de acesso (visibilidade). Veja os níveis de acesso com seus símbolos correspondentes:

- Público (+)
- Privado (-)
- Protegido (#)
- Pacote (~)
- Derivado (/)
- Estático (sublinhado)

2.9 Tipos de diagrama de classes

Relacionamento

As Classes costumam possuir relacionamento entre si, com o intuito de compartilhar informações e colaborarem umas com as outras para permitir a execução dos diversos processos executados pelo sistema.

- Associações
- Simples
- Herança/Generalização
- Agregação
- Composição
- Implementação (Interfaces)

Associação

Ela descreve um vínculo que ocorre entre classes — associação binária, mas é possível até mesmo que uma classe esteja vinculada a si própria, associação unária. Representamos as associações por meio de retas que ligam as classes envolvidas.

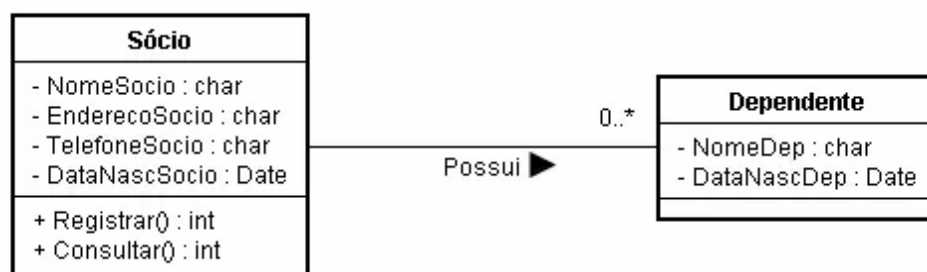


Figura.2

Agregação

O objeto-pai poderá usar as informações do objeto agregado.

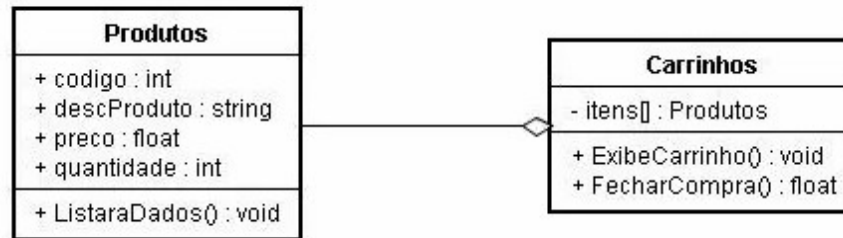


Figura.3

Composição

Pode-se dizer que composição é uma variação da agregação. Uma composição tenta representar também uma relação todo — parte. No entanto, na composição o objeto-pai (todo) é responsável por criar e destruir suas partes. Em uma composição um mesmo objeto-parte não pode se associar a mais de um objeto-pai. O todo não existe (ou não faz sentido) sem as partes ou, as partes não existem sem o todo.

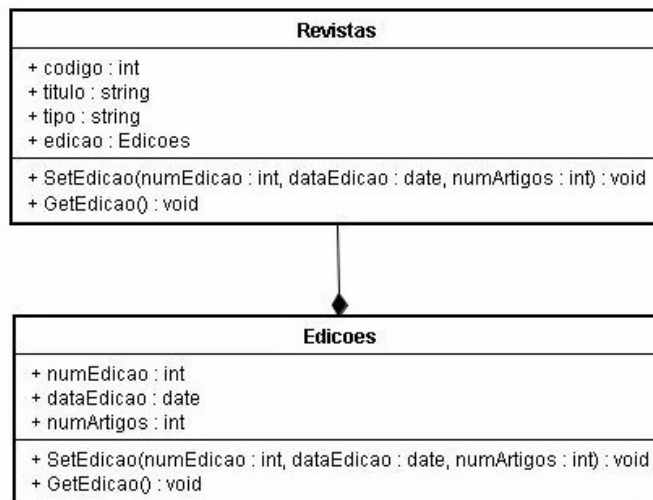


Figura.4

Especialização e Generalização

Atributos e métodos definidos na classe-mãe são herdados pelas classes-filhas.

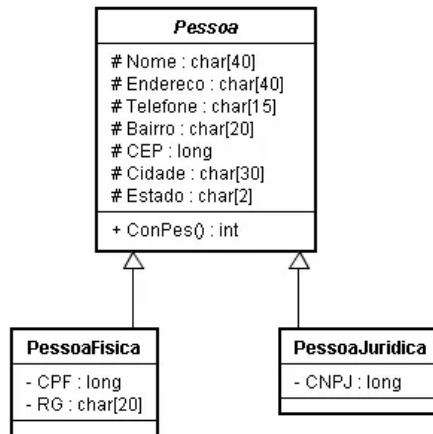


Figura.5

Interfaces

São elementos do modelo que definem conjuntos de operações que outros elementos do modelo, como classes ou componentes devem implementar.

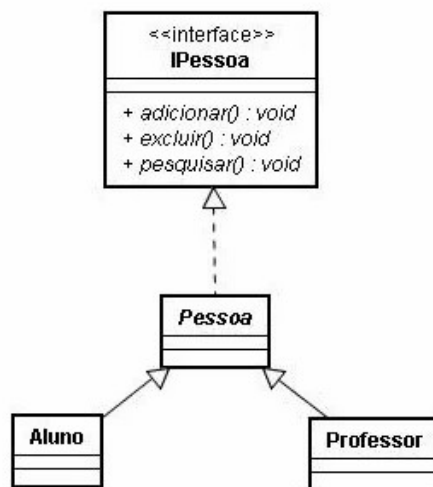


Figura.6

3. MÉTODOS

3.1 Diagrama de classe para um sistema de locação de veículos

Através dos Visual Paradigm foi realizado um diagrama de classes para um sistema de locação de veículos, conforme a **figura.7** abaixo.

Neste tipo de diagrama de classe foi utilizado um relacionamento de **associação** entre as classes (locação, automóvel, modelo e marca juntamente com a interface e a controladora).

Nos **atributos** e **métodos** com acesso **público (+)**, foi utilizado as seguintes variáveis para uma linguagem em C.

- **String** que é qualquer sequência de 4 ou mais caracteres imprimíveis que terminam com uma nova-linha ou um caractere nulo.
- **Int** que armazena valores inteiros, sem casas decimais.
- **Long** que representa um número inteiro.
- **Double** que é capaz de armazenar um número maior de casas decimais.
- **Void** que especifica que a função não retorna um valor.

Foi utilizado também algumas notações possíveis de relacionamento de uma classe para outra como:

- 1..* - relação um para muitos.
- 1 - relação um para um.
- 0..* - relação de zero para muitos.

3.2 Sistema de locação de veículos no Visual Paradigm

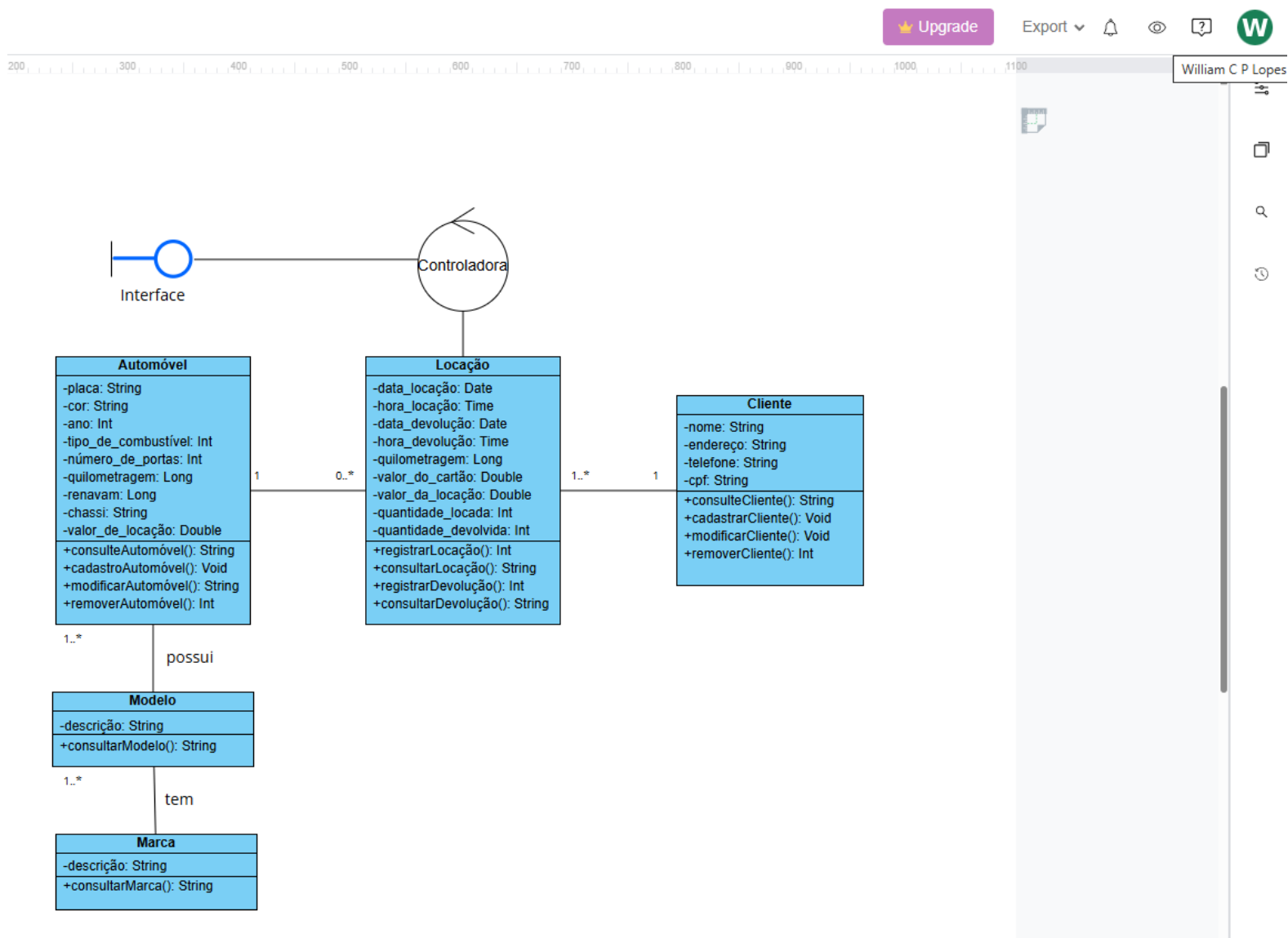


Figura.7

4. RESULTADOS

Ao desenvolver este diagrama pode-se perceber facilidade na utilização do software Visual Paradigm em criar as classes ou qualquer outro diagrama.

Dentro do diagrama de classes pode-se compreender as funcionalidades e interações com um sistema , no caso da atividade , um cliente com uma locação de veículos.

5. CONCLUSÃO

Ao final desta atividade pode-se concluir que ao utilizar as ferramentas do visual paradigm, obtive uma facilidade em aprender e desenvolver um diagrama em uma forma dinâmica e versátil utilizando alguns conceitos básicos, porém , eficientes para o objetivo proposto que era um diagrama de classe para um sistema de locação de veículos.

6. REFERÊNCIAS

<https://www.lucidchart.com/pages/pt>

<https://online.visual-paradigm.com/pt/>